

COMP3026E Distributed Systems

12. Concurrency Control

James Jianqiao Yu

Harbin Institute of Technology, Shenzhen

▶ Lets Scale Strong Consistency!

1. Atomic Commit
 - Two-phase commit (2PC)
2. Serializability
 - Strict serializability
3. Concurrency Control
 - Two-phase locking (2PL)
 - Optimistic concurrency control (OCC)

Atomic Commit



- Atomic: All or nothing
- Either all participants do something (commit) or no participant does anything (abort)
- Common use: commit a transaction that updates data on different shards

Transaction Examples

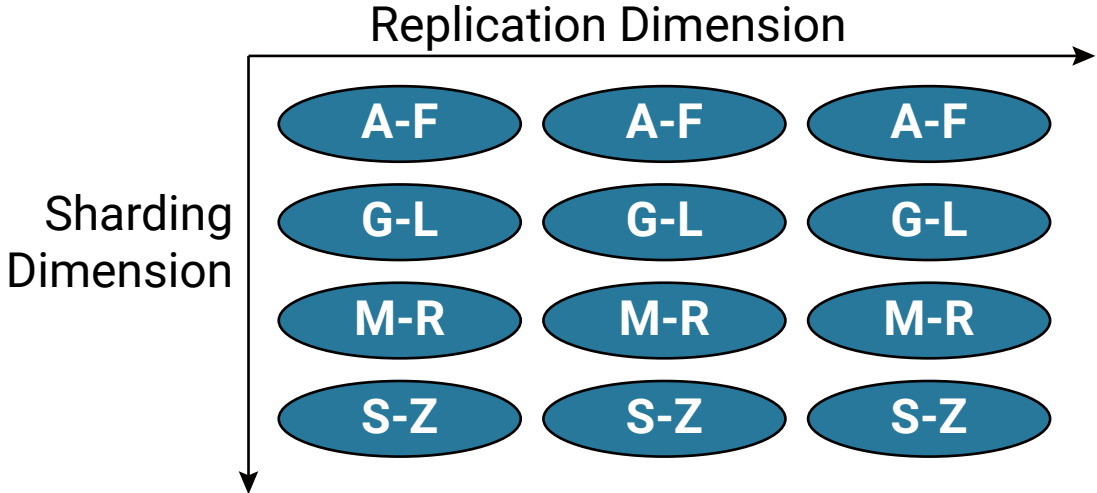
- Bank account transfer
 - Alice -= \$100
 - Bob += \$100
- Maintaining symmetric relationships
 - Bob `FriendOf` Alice
 - Alice `FriendOf` Bob
- Order product
 - Charge customer card
 - Decrement stock
 - Ship stock

► Relationship with Replication

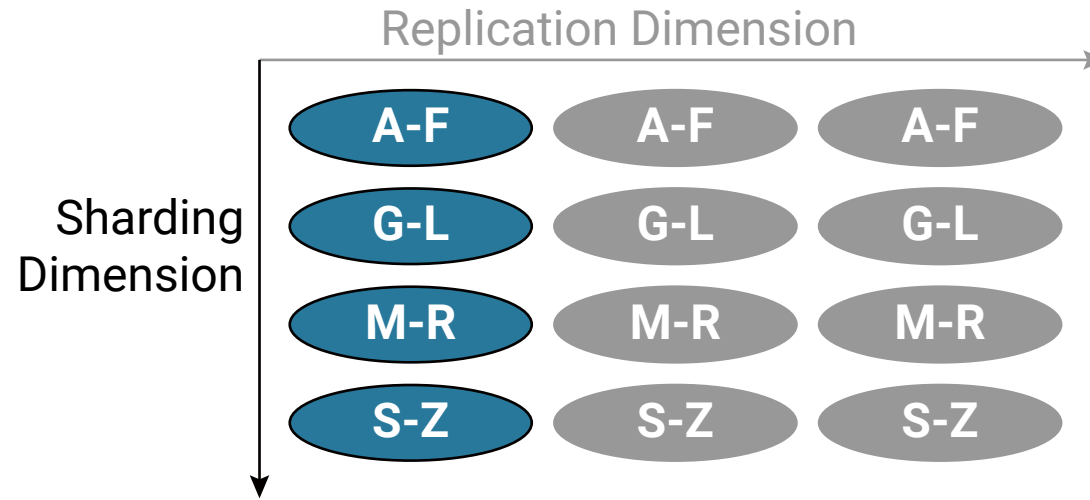
- Replication (e.g., RAFT) is about doing the **same** thing multiple places to provide fault tolerance
- Sharding is about doing **different** things multiple places for scalability
- Atomic commit is about doing **different** things in **different** places together



Relationship with Replication



Focus on Sharding for Today



- Atomic: All or nothing
- Either all participants do something (commit) or no participant does anything (abort)
- Atomic commit accomplished with **two-phase commit protocol (2PC)**

Two-Phase Commit



- **Phase 1**
 - Coordinator sends *Prepare* requests to all participants
 - Each participant votes yes or no
 - Sends yes or no vote back to coordinator
 - Typically acquires locks if they vote yes

- **Phase 1**
 - Coordinator sends *Prepare* requests to all participants
 - Each participant votes yes or no
 - Sends yes or no vote back to coordinator
 - Typically acquires locks if they vote yes
- **Coordinator** inspects all votes
 - If all yes, then commit
 - If any no, then abort

Two-Phase Commit



- **Phase 1**

- Coordinator sends *Prepare* requests to all participants
- Each participant votes yes or no
 - Sends yes or no vote back to coordinator
 - Typically acquires locks if they vote yes

- **Coordinator** inspects all votes

- If all yes, then commit
- If any no, then abort

- **Phase 2**

- Coordinator sends *Commit* or *Abort* to all participants
- If commit, each participant does something
- Each participant releases locks
- Each participant sends an *Ack* back to the coordinator

Unilateral Abort



- **Any** participant can cause an abort
- With 100 participants, if 99 vote yes and 1 votes no => abort!
- Common reasons to abort:
 - Cannot acquire required lock
 - No memory or disk space available to do write
 - Transaction constraint fails
 - (e.g., Alan does not have \$100)
- Q: Why do we want unilateral abort for atomic commit?

- Unilateral abort allows participants to avoid blocking other transactions
 - A participant can decide to abort on its own, without first getting permission from all others
 - Preserves atomicity by ensuring the transaction does not partially commit

- Unilateral abort allows participants to avoid blocking other transactions
 - A participant can decide to abort on its own, without first getting permission from all others
 - Preserves atomicity by ensuring the transaction does not partially commit
- **Commit must be carefully coordinated**
- **Abort is safe to do conservatively**

We want unilateral abort because uncertainty should safely lead to cancellation, not to a possibly inconsistent commit.

Atomic Commit: Summary



- All-or-nothing
- Unilateral abort
- Two-phase commit
 - Prepare -> Commit/abort

▶ Lets Scale Strong Consistency!

1. Atomic Commit
 - Two-phase commit (2PC)
2. **Serializability**
 - **Strict serializability**
3. Concurrency Control
 - Two-phase locking (2PL)
 - Optimistic concurrency control (OCC)

Two Concurrent Transactions



```
1 transaction transfer(A, B):  
2   begin_tx  
3     a <- read(A)  
4     if a < 10 then abort_tx  
5     else write(A, a+10)  
6         b <- read(B)  
7         write(B, b+10)  
8         commit_tx
```

Two Concurrent Transactions

```
1 transaction sum(A, B):  
2 begin_tx  
3   a <- read(A)  
4   b <- read(B)  
5   print a + b  
6 commit tx
```

```
1 transaction transfer(A, B):  
2 begin_tx  
3   a <- read(A)  
4   if a < 10 then abort_tx  
5   else write(A, a-10)  
6       b <- read(B)  
7       write(B, b+10)  
8       commit tx
```

Isolation Between Transactions

- **Isolation**: sum appears to happen either completely before or completely after transfer
 - i.e., it appears that all operations of a transaction happened together
- **Schedule** for transactions is an ordering of the operations performed by those transactions

Problem from Concurrent Execution

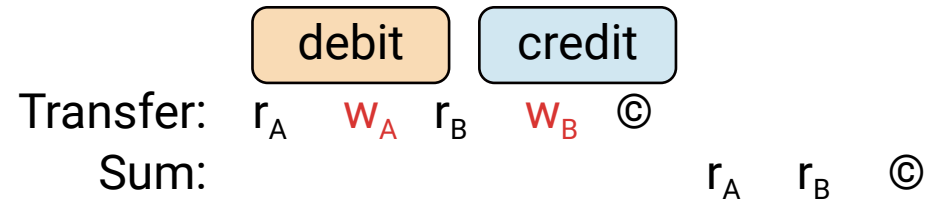
- Serial execution of transactions—transfer then sum:

Transfer: r_A w_A r_B w_B ©

Sum: r_A r_B ©

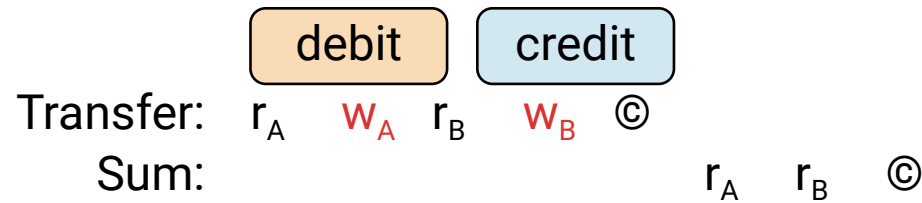
Problem from Concurrent Execution

- Serial execution of transactions—transfer then sum:

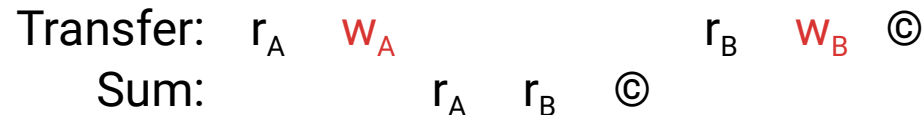


Problem from Concurrent Execution

- Serial execution of transactions—transfer then sum:

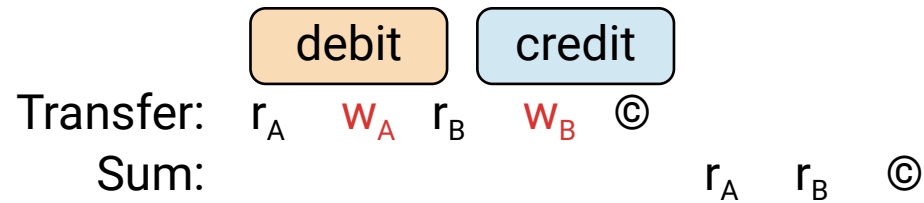


- Concurrent execution can result in state that differs from any serial execution:

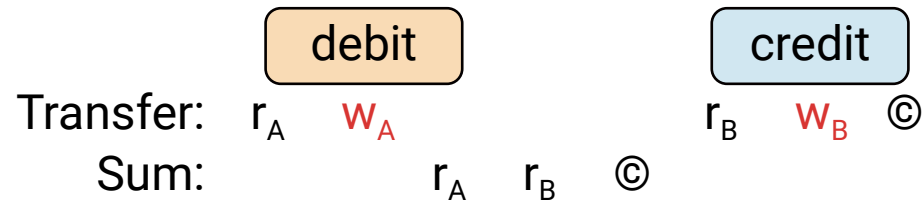


Problem from Concurrent Execution

- Serial execution of transactions—transfer then sum:



- Concurrent execution can result in state that differs from any serial execution:



Isolation Between Transactions

- Isolation: sum appears to happen either completely before or completely after transfer
 - i.e., it appears that all operations of a transaction happened together
- Given a schedule of operations:
 - *Is that schedule in some way “equivalent” to a serial execution of transactions?*

Equivalence of Schedules

- Two operations from different transactions are **conflicting** if:
 1. They read and write to the same data item
 2. They write and write to the same data item

Read/Write-write conflict because changing their order changes the result

Equivalence of Schedules

- Two operations from different transactions are **conflicting** if:
 1. They read and write to the same data item
 2. They write and write to the same data item

Read/Write-write conflict because changing their order changes the result

- Two schedules are **equivalent** if:
 1. They contain the same transactions and operations
 2. They **order** all **conflicting** operations of non-aborting transactions in the **same way**

- A schedule is **serializable** if it is equivalent to some serial schedule
 - i.e., non-conflicting operations can be reordered to get a serial schedule

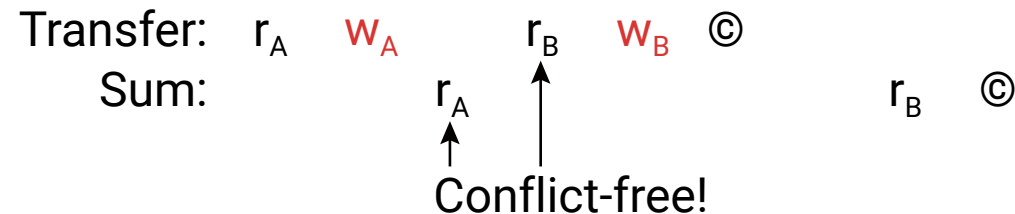
A Serializable Schedule

- A schedule is **serializable** if it is equivalent to some serial schedule
 - i.e., non-conflicting operations can be reordered to get a serial schedule

Transfer: r_A w_A r_B w_B ©
Sum: r_A r_B ©

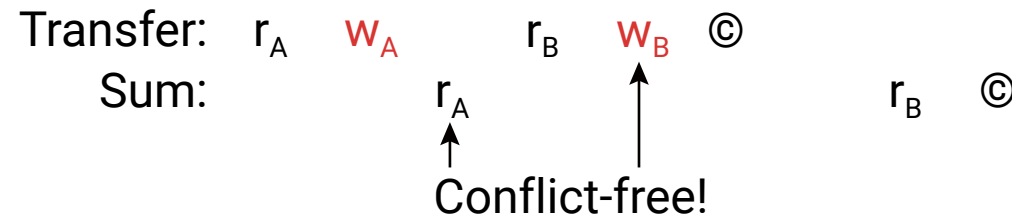
A Serializable Schedule

- A schedule is **serializable** if it is equivalent to some serial schedule
 - i.e., non-conflicting operations can be reordered to get a serial schedule



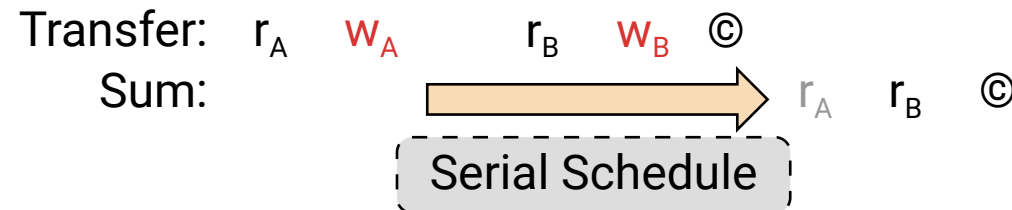
A Serializable Schedule

- A schedule is **serializable** if it is equivalent to some serial schedule
 - i.e., non-conflicting operations can be reordered to get a serial schedule



A Serializable Schedule

- A schedule is **serializable** if it is equivalent to some serial schedule
 - i.e., non-conflicting operations can be reordered to get a serial schedule



A Non-Serializable Schedule

- A schedule is **serializable** if it is equivalent to some serial schedule
 - i.e., non-conflicting operations can be reordered to get a serial schedule

Transfer: r_A w_A r_B w_B ©
Sum: r_A r_B ©

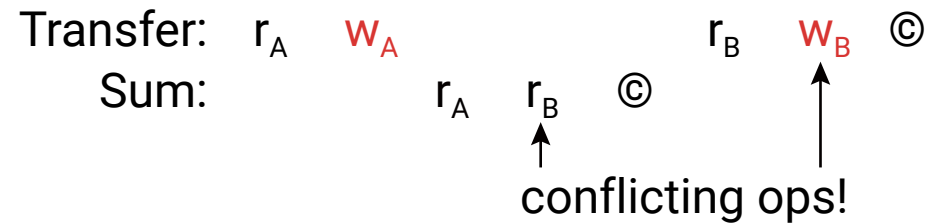
A Non-Serializable Schedule

- A schedule is **serializable** if it is equivalent to some serial schedule
 - i.e., non-conflicting operations can be reordered to get a serial schedule

Transfer: r_A w_A r_B w_B ©
Sum: r_A r_A r_B ©
 ↑ ↑
 conflicting ops!

A Non-Serializable Schedule

- A schedule is **serializable** if it is equivalent to some serial schedule
 - i.e., non-conflicting operations can be reordered to get a serial schedule



A Non-Serializable Schedule

- A schedule is **serializable** if it is equivalent to some serial schedule
 - i.e., non-conflicting operations can be reordered to get a serial schedule

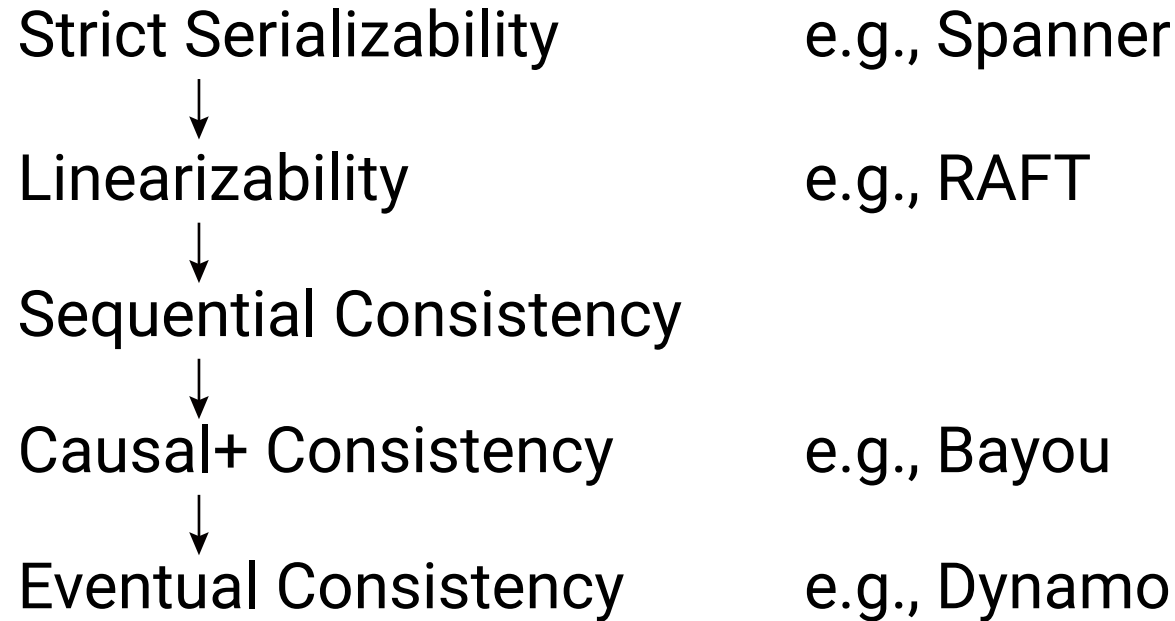
Transfer: r_A w_A r_B w_B ©
Sum: r_A r_B ©

But in a serial schedule, sum's reads either both before w_A or both after w_B

Linearizability vs. Serializability

- **Linearizability**: a guarantee about single operations on single objects
 - Once write completes, all reads that begin later should reflect that write
- **Serializability** is guarantee about transactions over one or more objects
 - Doesn't impose real-time constraints
- **Strict Serializability** = **Serializability** + real-time ordering
 - Intuitively **Serializability** + **Linearizability**
 - We'll stick with only **Strict Serializability** for this class

Consistency Hierarchy



▶ Lets Scale Strong Consistency!

1. Atomic Commit
 - Two-phase commit (2PC)
2. Serializability
 - Strict serializability
3. **Concurrency Control**
 - **Two-phase locking (2PL)**
 - **Optimistic concurrency control (OCC)**

Concurrency Control

- Concurrent execution can violate serializability
- We need to control that concurrent execution so we do things a single machine executing transactions one at a time would
 - **Concurrency control**

Concurrency Control Strawman #1



- Big global lock
 - Acquire the lock when transaction starts
 - Release the lock when transaction ends
- Provides strict serializability
 - Just like executing transaction one by one because we are doing exactly that
- No concurrency at all
 - Terrible for performance: one transaction at a time

Locking

- Locks maintained on each shard
 - Transaction requests lock for a data item
 - Shard grants or denies lock
- Lock types
 - **Shared**: Need to have before read object
 - **Exclusive**: Need to have before write object

	Shared (S)	Exclusive (X)
Shared (S)	Yes	No
Exclusive (X)	No	No

Concurrency Control Strawman #2



- Grab locks independently, for each data item (e.g., bank accounts A and B)

Transfer: $\blacktriangle_A r_A w_A \blacktriangle_A$
Sum:

$\blacktriangle_B r_B w_B \blacktriangle_B \textcircled{C}$

$\blacktriangle / \triangle = \text{eXclusive- / Shared-lock}$ $\blacktriangle / \triangle = \text{X- / S-unlock}$

Concurrency Control Strawman #2

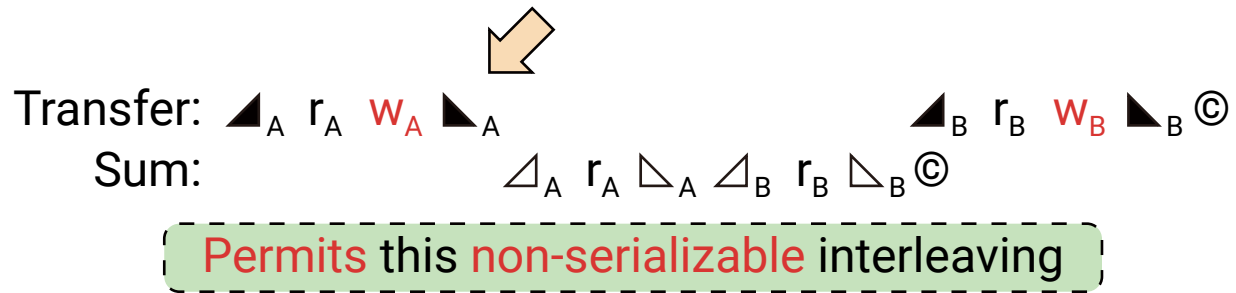
- Grab locks independently, for each data item (e.g., bank accounts A and B)

Transfer: $\blacktriangle_A r_A w_A \blacktriangle_A$ $\blacktriangle_B r_B w_B \blacktriangle_B \textcircled{C}$
 Sum: $\triangle_A r_A \triangle_A \triangle_B r_B \triangle_B \textcircled{C}$

$\blacktriangle / \triangle = \text{eXclusive- / Shared-lock}$ $\blacktriangle / \triangle = \text{X- / S-unlock}$

Concurrency Control Strawman #2

- Grab locks independently, for each data item (e.g., bank accounts A and B)



$\blacktriangle / \triangle = \text{eXclusive- / Shared-lock}$ $\blacktriangle / \triangle = \text{X- / S-unlock}$

Two-Phase Locking (2PL)

- 2PL rule: Once a transaction has released a lock it is not allowed to obtain any other locks
 - Growing phase: transaction acquires locks
 - Shrinking phase: transaction releases locks
- In practice:
 - Growing phase is the entire transaction
 - Shrinking phase is during commit

2PL Provide Strict Serializability

- 2PL rule: Once a transaction has released a lock it is not allowed to obtain any other locks

Transfer: $\blacktriangle_A r_A w_A \blacktriangle_A$ $\blacktriangle_B r_B w_B \blacktriangle_B \textcircled{C}$
 Sum: $\triangle_A r_A \triangle_A \triangle_B r_B \triangle_B \textcircled{C}$

$\blacktriangle / \triangle = \text{eXclusive- / Shared-lock}$ $\blacktriangle / \triangle = \text{X- / S-unlock}$

2PL Provide Strict Serializability

- 2PL rule: Once a transaction has released a lock it is not allowed to obtain any other locks



- Q: What is not allowed by 2PL here?
 - Sum can't grab a new lock on b once it releases its lock on A
 - Transfer can't grab a new lock on B once it released its lock on A

$\blacktriangle / \triangle = \text{eXclusive- / Shared-lock}$ $\blacktriangle / \triangle = \text{X- / S-unlock}$

2PL and Transaction Concurrency

- 2PL rule: Once a transaction has released a lock it is not allowed to obtain any other locks

Transfer: $\triangle_A \ r_A \quad \blacktriangle_A \ W_A \triangle_B \ r_B \quad \blacktriangle_B \ W_B \quad \text{※} \quad \text{©}$
 Sum: $\triangle_A \ r_A \quad \triangle_B \ r_B \quad \text{※} \quad \text{©}$

2PL permits this serializable, interleaved schedule

$\blacktriangle / \triangle = \text{eXclusive- / Shared-lock}$ $\blacktriangle / \triangle = \text{X- / S-unlock}$ $\text{※} = \text{release all}$



2PL Doesn't Exploit All Concurrency Opportunities



- 2PL rule: Once a transaction has released a lock it is not allowed to obtain any other locks

Transfer: r_A w_A r_B w_B ©
Sum: r_A r_B ©

2PL precludes this serializable, interleaved schedule

Issues with 2PL

- What do we do if a lock is unavailable?
 - Give up immediately?
 - Wait forever?
- Waiting for a lock can result in **deadlock**
 - Transfer has A locked, waiting on B
 - Sum has B locked, waiting on A
- Many different ways to detect and deal with deadlocks

More Concurrency Control Algorithms

- Optimistic Concurrency Control (OCC)
- Multi-Version Concurrency Control (MVCC)



Optimistic Concurrency Control (OCC)



Idea Let transactions run freely first, and check for conflicts only before commit

1. **Read**: transaction reads database items and writes updates to a private workspace
 2. **Validation**: before commit, check whether this transaction conflicts with other committed/validating transactions
 3. **Write**: if validation succeeds, copy private updates to the database; otherwise abort and restart
- Why “optimistic”? Assumes conflicts are **rare**



Optimistic Concurrency Control (OCC)



- **Pros:** high concurrency, no lock blocking, no deadlock
- **Cons:** wasted work on aborts, poor under high contention, restarts may be frequent
- Best use cases
 - Read-heavy workloads
 - Low-conflict environments
 - Systems where blocking is expensive



Multi-Version Concurrency Control (MVCC)



Idea Keep **multiple versions** of the same data item so reads do not have to wait for writes.

- When a transaction writes **X**, it creates a new version of **X**
 - Old version is kept temporarily for transactions that started earlier
 - A transaction reads the version that matches its timestamp / snapshot
- Why?
 - Readers do not block writers
 - Writers do not block readers
 - Each transaction sees a consistent snapshot of the database



Multi-Version Concurrency Control (MVCC)



- **Advantages**

- High read concurrency
- Good performance for read-heavy workloads
- Fewer lock waits than single-version schemes

- **Challenges**

- Need extra storage for old versions
- System must clean up obsolete versions
- Write-write conflicts still need control



© 2026 James Jianqiao Yu
Harbin Institute of Technology, Shenzhen

All rights reserved.

These lecture materials are intended solely for educational use in the course *Distributed Systems*. Permission is granted to students enrolled in the course to reproduce these materials, in whole or in part, for personal, non-commercial use only (including printing and photocopying). Any further reproduction or distribution, including but not limited to public posting on websites, social media, forums, or file-sharing platforms, distribution to non-enrolled individuals, or any commercial use, is prohibited without the prior written permission of the author.

This slide deck is adapted from Princeton University's *Distributed Systems* course materials, originally created by Michael Freedman, Kyle Jamieson, and Wyatt Lloyd, and licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported (CC BY-NC-SA 3.0) License. The original authors and source are gratefully acknowledged.